CENTRO DE ENSINO SUPERIOR DE MARINGÁ – UNICESUMAR
CURSO DE ENGENHARIA DE SOFTWARE
DISCIPLINA: ENGENHARIA DE SOFTWARE

FERNANDO CÉSAR DE OLIVEIRA JUNIOR
GUSTAVO STARKE BRANDT

DIAGRAMA DE CLASSES DE UMA CLÍNICA VETERINÁRIA

TEMA: DIAGRAMA DE CLASSES – GENERALIZAÇÃO, CLASSE ABSTRATA E POLIMORFISMO

CIDADE
2026

SUMÁRIO

1 INTRODUÇÃO..... 3
2 FUNDAMENTAÇÃO..... 3
2.1 Diagrama de classes..... 3
2.2 Generalização e herança..... 4
2.3 Classe abstrata e método abstrato..... 4
2.4 Sobrescrita de métodos..... 5
2.5 Polimorfismo..... 5
3 MODELAGEM DO CENÁRIO..... 5
3.1 Regras de negócio..... 5
3.2 Diagrama de classes completo..... 6
3.3 Hierarquia de atendimentos..... 8
3.4 Polimorfismo na fatura..... 9
4 ANÁLISE..... 10
5 CONCLUSÃO..... 11
REFERÊNCIAS..... 11

1 INTRODUÇÃO

Este trabalho foi feito para a disciplina de Engenharia de Software e trata do diagrama de classes da UML (*Unified Modeling Language*), com foco em três assuntos: generalização, classe abstrata e polimorfismo. Os três estão ligados entre si. A generalização cria a hierarquia de classes, a classe abstrata define o que todas as classes da hierarquia precisam ter e o polimorfismo permite tratar objetos de classes diferentes da mesma forma.

O objetivo do trabalho é entender esses conceitos e aplicá-los na modelagem de um sistema, mostrando como cada um aparece no diagrama e explicando o motivo das decisões tomadas. Os diagramas foram feitos na ferramenta Mermaid, que gera diagramas UML a partir de um código em texto, e os arquivos-fonte foram publicados no repositório da dupla no GitHub.

O cenário escolhido foi o sistema de atendimentos e faturamento de uma clínica veterinária. Na clínica, os tutores levam seus animais para consultas, vacinas, exames e cirurgias, e cada tipo de atendimento é cobrado de um jeito diferente. No fim, o tutor recebe uma fatura com todos os atendimentos e o total a pagar. Escolhemos esse cenário porque os atendimentos têm várias informações em comum, como a data, o animal e o veterinário, mas cada tipo tem a sua própria regra de cobrança. É exatamente o tipo de situação em que herança e polimorfismo fazem sentido.

O trabalho está organizado da seguinte forma: a seção 2 apresenta os conceitos do tema; a seção 3 mostra as regras do cenário e os diagramas; a seção 4 discute as decisões de modelagem e as dificuldades que tivemos; e a seção 5 traz a conclusão.

2 FUNDAMENTAÇÃO

2.1 Diagrama de classes

O diagrama de classes mostra a estrutura de um sistema: quais classes existem, que dados cada uma guarda, o que cada uma faz e como elas se relacionam. Guedes (2011) aponta o diagrama de classes como um dos mais importantes e mais usados da UML, já que ele serve de base para outros diagramas e para a programação do sistema.

Cada classe é desenhada como um retângulo com três partes: nome, atributos e métodos. Antes de cada membro vem o símbolo de visibilidade: + público, - privado, # protegido (acessível pela própria classe e pelas subclasses) e ~ pacote. Depois do nome vem o tipo, separado por dois-pontos, como em - nome: String. Nos métodos, o tipo indicado é o do valor de retorno.

2.2 Generalização e herança

A generalização é o relacionamento entre uma classe mais geral, chamada de superclasse, e uma ou mais classes mais específicas, chamadas de subclasses. A subclasse herda os atributos e os métodos da superclasse e pode acrescentar os seus. Com isso, o que é comum fica escrito em um lugar só, e cada subclasse cuida apenas do que é diferente. Pressman (2011) trata a herança como uma das características mais importantes da orientação a objetos justamente por permitir esse reaproveitamento.

No diagrama, a generalização é uma linha cheia com um triângulo vazado do lado da superclasse. Uma forma simples de conferir se a herança está correta é o teste do “é um”: uma Vacina é um Procedimento, e um Procedimento é um Atendimento. Se a frase não faz sentido, provavelmente não é herança. A hierarquia pode ter vários níveis, em que uma subclasse é superclasse de outras.

2.3 Classe abstrata e método abstrato

Uma classe abstrata é uma classe da qual não se criam objetos. Ela existe para servir de modelo para as subclasses, reunindo o que todas têm em comum. Deitel e Deitel (2017) explicam que esse tipo de classe é usado como superclasse em hierarquias de herança e que, por ser incompleta, não pode ser instanciada.

A classe abstrata pode ter atributos e métodos normais e também métodos abstratos. Um método abstrato tem só a assinatura (nome, parâmetros e tipo de retorno), sem implementação. Ele funciona como uma obrigação: toda subclasse concreta, ou seja, que não é abstrata, precisa implementar esse método. Uma subclasse que também é abstrata pode deixar essa tarefa para as classes abaixo dela.

Na UML, a classe abstrata é indicada com o nome em itálico ou com o estereótipo «abstract» acima do nome, e o método abstrato aparece em itálico. No

Mermaid, ferramenta usada neste trabalho, a classe recebe o estereótipo e o método abstrato fica em itálico.

2.4 Sobrescrita de métodos

A sobrescrita acontece quando a subclasse declara um método com a mesma assinatura de um método da superclasse e dá a ele uma implementação própria. No diagrama, o método sobrescrito aparece de novo dentro da subclasse. Ela é diferente da sobrecarga, em que métodos da mesma classe têm o mesmo nome e parâmetros diferentes.

2.5 Polimorfismo

Polimorfismo significa “muitas formas”. Na orientação a objetos, é a possibilidade de uma mesma chamada de método ter comportamentos diferentes conforme o objeto que a recebe. Isso funciona porque uma variável do tipo da superclasse pode guardar um objeto de qualquer subclasse. Quando um método sobrescrito é chamado, o que executa é a versão da classe real do objeto, e não a do tipo da variável.

Segundo Deitel e Deitel (2017), o polimorfismo permite escrever programas que tratam os objetos de uma hierarquia de forma geral, como se todos fossem da superclasse, sem precisar testar o tipo de cada um. A vantagem é que um tipo novo pode ser incluído depois sem alterar o código que já usa a superclasse.

O polimorfismo não tem um símbolo próprio no diagrama de classes. Ele aparece pela combinação de três coisas: uma classe que se relaciona com a superclasse abstrata, um método abstrato nessa superclasse e as subclasses sobrescrevendo esse método. A seção 3.4 mostra esse caso no cenário da clínica.

3 MODELAGEM DO CENÁRIO

3.1 Regras de negócio

O sistema modelado controla os atendimentos de uma clínica veterinária e a cobrança deles. As regras definidas pela dupla foram as seguintes, sendo que as

regras c) a f) são as que justificam a herança, porque todos são atendimentos, mas cada tipo calcula o valor de um jeito:

a) cada tutor pode ter um ou mais animais cadastrados, e cada animal tem um único tutor responsável;
b) todo atendimento é feito por um veterinário, para um animal, em uma data;
c) a consulta tem valor fixo, e o retorno feito em até 15 dias é gratuito;
d) a vacina cobra a taxa de aplicação mais o valor da dose, e a dose de reforço cobra só a dose;
e) o exame cobra o valor do exame mais o material, com acréscimo de 30% quando é urgente;
f) a cirurgia cobra o valor do procedimento, o material, a anestesia e as diárias de internação;
g) a clínica emite para o tutor uma fatura com os atendimentos realizados, e o total da fatura é a soma do valor de cada atendimento.

3.2 Diagrama de classes completo

A Figura 1 apresenta o diagrama de classes do sistema, com dez classes. As classes Tutor, Animal, Veterinario e Fatura representam as pessoas, os animais e a cobrança. As outras seis formam a hierarquia de herança que começa em Atendimento, detalhada na seção 3.3.

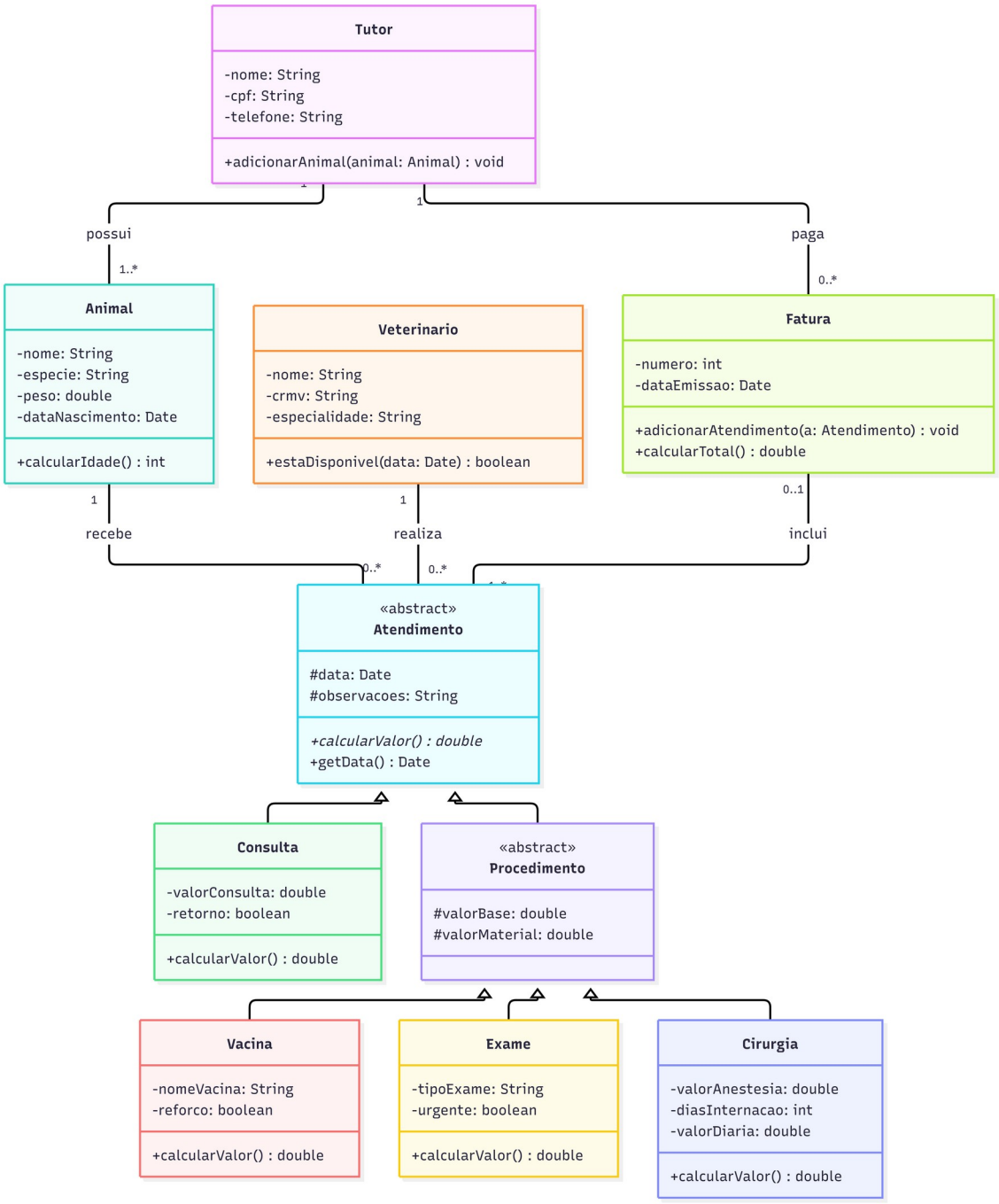
Além da herança, o diagrama tem cinco associações. Cada uma recebeu um nome e as multiplicidades nas duas pontas, e pode ser lida nos dois sentidos:

a) Tutor possui Animal: um tutor tem um ou mais animais (1..*), e cada animal pertence a exatamente um tutor (1);
b) Animal recebe Atendimento: um animal pode ter nenhum ou vários atendimentos (0..*), pois pode estar cadastrado sem ter sido atendido, e cada atendimento é de um único animal (1);
c) Veterinario realiza Atendimento: um veterinário pode ter nenhum ou vários atendimentos (0..*), e cada atendimento é feito por um único veterinário (1);
d) Tutor paga Fatura: um tutor pode ter nenhuma ou várias faturas (0..*), e cada fatura é de um único tutor (1);

e) Fatura inclui Atendimento: uma fatura tem pelo menos um atendimento (1..*), e cada atendimento está em no máximo uma fatura (0..1), já que fica sem fatura enquanto não é cobrado.

As associações não foram repetidas como atributos dentro das classes. A Fatura, por exemplo, não tem um atributo atendimentos, porque a linha da associação já representa essa informação, e repetir os dois seria redundante.

Figura 1 – Diagrama de classes da clínica veterinária

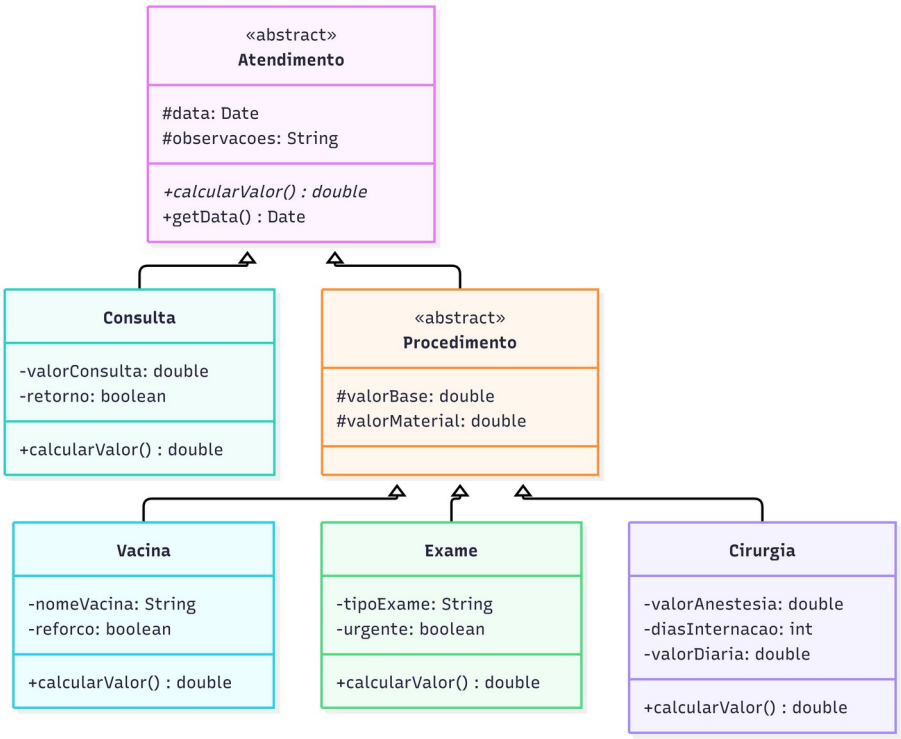


Fonte: elaborado pelos autores (2026)

3.3 Hierarquia de atendimentos

A Figura 2 mostra apenas a hierarquia de herança, para que os detalhes fiquem legíveis. Ela tem três níveis.

Figura 2 – Hierarquia de herança dos atendimentos



Fonte: elaborado pelos autores (2026)

No primeiro nível está Atendimento, que é abstrata. Ela guarda o que todo atendimento tem (data e observacoes) e declara o método abstrato calcularValor(), que obriga cada tipo a dizer quanto custa. Já o método getData() é concreto e é herdado sem mudança.

No segundo nível estão Consulta, que é concreta e já implementa calcularValor(), e Procedimento, que também é abstrata e reúne valorBase e valorMaterial, usados por vacinas, exames e cirurgias. Procedimento não implementa calcularValor(), o que é permitido por ser abstrata, e deixa essa obrigação para Vacina, Exame e Cirurgia, no terceiro nível.

Os atributos das superclasses são protegidos (#) porque as subclasses precisam deles no cálculo, e os das subclasses são privados (-). O método calcularValor() aparece nas quatro classes concretas como sobrescrita do método abstrato, e o Quadro 1 resume o cálculo de cada uma.

Quadro 1 – Regra de cálculo de calcularValor() em cada classe concreta

Classe	**Cálculo feito por calcularValor()**	**Exemplo**
Consulta	valorConsulta; se for retorno, o valor é zero	Consulta: R$ 150,00; retorno: R$ 0,00
Vacina	valorBase (taxa de aplicação) + valorMaterial (dose); se for reforço, só valorMaterial	30 + 90 = R$ 120,00; reforço: R$ 90,00
Exame	valorBase + valorMaterial; se for urgente, o resultado é multiplicado por 1,3	(60 + 20) × 1,3 = R$ 104,00
Cirurgia	valorBase + valorMaterial + valorAnestesia + diasInternacao × valorDiaria	600 + 150 + 250 + 2 × 120 = R$ 1.240,00

Fonte: elaborado pelos autores (2026)

3.4 Polimorfismo na fatura

O polimorfismo do cenário está na classe Fatura. Ela se associa com Atendimento, que é a superclasse abstrata, e não com Consulta, Vacina, Exame ou Cirurgia. Para a Fatura, todos são apenas atendimentos. O Quadro 2 mostra como isso fica em código Java.

Quadro 2 – Trecho de código em Java do polimorfismo na fatura

```
public abstract class Atendimento {
    protected Date data;
    public abstract double calcularValor();
}

public class Exame extends Procedimento {
    private boolean urgente;

    @Override
    public double calcularValor() {
        double valor = valorBase + valorMaterial;
        if (urgente) {
            valor = valor * 1.3;
        }
        return valor;
    }
}

public class Fatura {
    private List<Atendimento> atendimentos = new ArrayList<>();

    public double calcularTotal() {
        double total = 0;
        for (Atendimento a : atendimentos) {
            total += a.calcularValor(); // cada tipo calcula do seu jeito
        }
        return total;
    }
}
```

Fonte: elaborado pelos autores (2026)

No método calcularTotal(), a linha total += a.calcularValor() é sempre a mesma, mas o que ela executa depende do objeto guardado em a. Em uma fatura com uma consulta, uma vacina e um exame urgente, a mesma linha roda três versões diferentes do método e soma R$ 150,00, R$ 120,00 e R$ 104,00, chegando ao total de R$ 374,00. A Fatura não precisa de nenhum if para descobrir o tipo de cada atendimento. Repare também que a associação com Atendimento vira, no código, a lista atendimentos, que no diagrama é representada só pela linha da associação.

Outra vantagem aparece se a clínica quiser cobrar um serviço novo, como a aplicação de microchip: basta criar uma subclasse de Procedimento com a sua versão de calcularValor(), sem alterar a Fatura.

4 ANÁLISE

A primeira decisão de modelagem foi deixar Atendimento como classe abstrata. No sistema não existe um atendimento “genérico”: todo atendimento é uma consulta, uma vacina, um exame ou uma cirurgia. Um objeto Atendimento puro não saberia calcular o próprio valor.

A segunda foi criar o nível intermediário Procedimento. Sem ele, os atributos valorBase e valorMaterial teriam que ser repetidos em Vacina, Exame e Cirurgia. A Consulta ficou fora de Procedimento porque não usa material e o seu valor é fixo. Esse nível a mais também garantiu os três níveis de herança pedidos no tema.

A terceira foi não criar subclasses para Animal. Pensamos em fazer Cachorro e Gato herdando de Animal, mas no sistema os dois se comportam da mesma forma, e a diferença entre eles é só o valor do atributo especie. Concluímos que a herança só vale a pena quando as subclasses têm comportamento diferente ou dados próprios.

A quarta foi ligar a Fatura direto em Atendimento. Com uma associação para cada subclasse, o cálculo do total teria que testar o tipo de cada atendimento, e a Fatura precisaria mudar sempre que surgisse um tipo novo.

Entre as dificuldades, a principal foi decidir onde parar a hierarquia. Pensamos em dividir Exame em ExameLaboratorial e ExameDeImagem, mas a cobrança dos dois seria igual, então a divisão não acrescentaria nada. Também tivemos dúvida na multiplicidade entre Fatura e Atendimento e escolhemos 0..1 do lado da fatura, porque um atendimento existe antes de ser cobrado.

Outra dificuldade foi a notação na ferramenta. Na primeira versão do diagrama, só o método calcularValor() estava marcado como abstrato, e as classes Atendimento e Procedimento ficaram sem o estereótipo «abstract». O tipo de retorno dos métodos também foi escrito com dois-pontos, o que fez o Mermaid mostrar os dois-pontos duplicados. As duas coisas foram corrigidas na versão final.

5 CONCLUSÃO

O objetivo deste trabalho era estudar generalização, classe abstrata e polimorfismo no diagrama de classes e aplicá-los em um cenário real. Com o sistema da clínica veterinária, foi possível mostrar uma hierarquia de três níveis, duas classes abstratas, um método abstrato sobrescrito em quatro classes e um exemplo de polimorfismo no cálculo da fatura.

O principal aprendizado foi perceber que esses conceitos não servem só para economizar código. Quando a hierarquia é bem pensada, o sistema fica mais fácil de mudar: um novo tipo de atendimento entra como uma nova subclasse, sem mexer no que já funciona. Também aprendemos que nem tudo deve virar herança, como no caso de Animal, e que a pergunta certa é se as classes realmente se comportam de forma diferente.

A maior dificuldade foi decidir a estrutura da hierarquia e acertar a notação na ferramenta. Com as correções, o diagrama final atende aos itens pedidos no tema e ficou coerente com as regras da clínica.

REFERÊNCIAS

DEITEL, Paul; DEITEL, Harvey. **Java**: como programar. 10. ed. São Paulo: Pearson, 2017.

GUEDES, Gilleanes T. A. **UML 2**: uma abordagem prática. 2. ed. São Paulo: Novatec, 2011.

PRESSMAN, Roger S. **Engenharia de software**: uma abordagem profissional. 7. ed. Porto Alegre: AMGH, 2011.